**Algorithm Analysis Fundamentals**

Time complexity analysis of iterative and recursive algorithms; verify Big-O, Ω, Θ for sample programs

Student name : M.BalaSaravanan (192524456) Department : B.TECH AI&DS

Course name & code : Design and analysis of algorithm - CSA0618

# Experiment 1: Linear Search (Iterative)

- **Objective:** To analyze time complexity of linear search.
- **Problem Statement:** Write a Python program to search for a key in an array using linear search.
- **Sample Input:** [10, 25, 30, 45, 50], Key = 30
- **Sample Output:** Key found at index 2
- **Complexity:** Ω: O(1), O: O(n), Θ: Θ(n)
- **Explanation:** Demonstrates sequential scanning and inefficiency for large inputs.

**Code:**

```
#include<stdio.h>

int main(){

 int a[]={10,25,30,45,50},k=30,i;

 for(i=0;i<5;i++)

  if(a[i]==k){

   printf("Found at index %d",i);

   return 0;

  }

 printf("Not found");

}
```

Output:

```c
#include<stdio.h>
int main(){
 int a[]={10,25,30,45,50},k=30,i;
 for(i=0;i<5;i++)
  if(a[i]==k){printf("Found at index %d",i);return 0;}
 printf("Not found");
}
```

```
Found at index 2

...Program finished with exit code 0
Press ENTER to exit console.
```

# Experiment 2: Binary Search (Recursive)

- **Objective:** To compare recursive binary search complexity.
- **Problem Statement:** Implement recursive binary search in Python.
- **Sample Input:** [5, 10, 15, 20, 25], Key = 20
- **Sample Output:** Key found at index 3
- **Complexity:** Ω: O(1), O: O(log n), Θ: Θ(log n)
- **Explanation:** Shows divide-and-conquer and logarithmic efficiency.

**Code :**

```
#include<stdio.h>

int bs(int a[],int l,int h,int k){

 if(l>h)return -1;

 int m=(l+h)/2;

 if(a[m]==k)return m;

 if(k<a[m])return bs(a,l,m-1,k);

 return bs(a,m+1,h,k);

}

int main(){

 int a[]={5,10,15,20,25};

 printf("Found at index %d",bs(a,0,4,20));

}
```

Output :

```c
#include<stdio.h>
int bs(int a[],int l,int h,int k){
 if(l>h)return -1;
 int m=(l+h)/2;
 if(a[m]==k)return m;
 if(k<a[m])return bs(a,l,m-1,k);
 return bs(a,m+1,h,k);
}
int main(){
 int a[]={5,10,15,20,25};
 printf("Found at index %d",bs(a,0,4,20));
}
```

```
Found at index 3

...Program finished with exit code 0
Press ENTER to exit console.
```

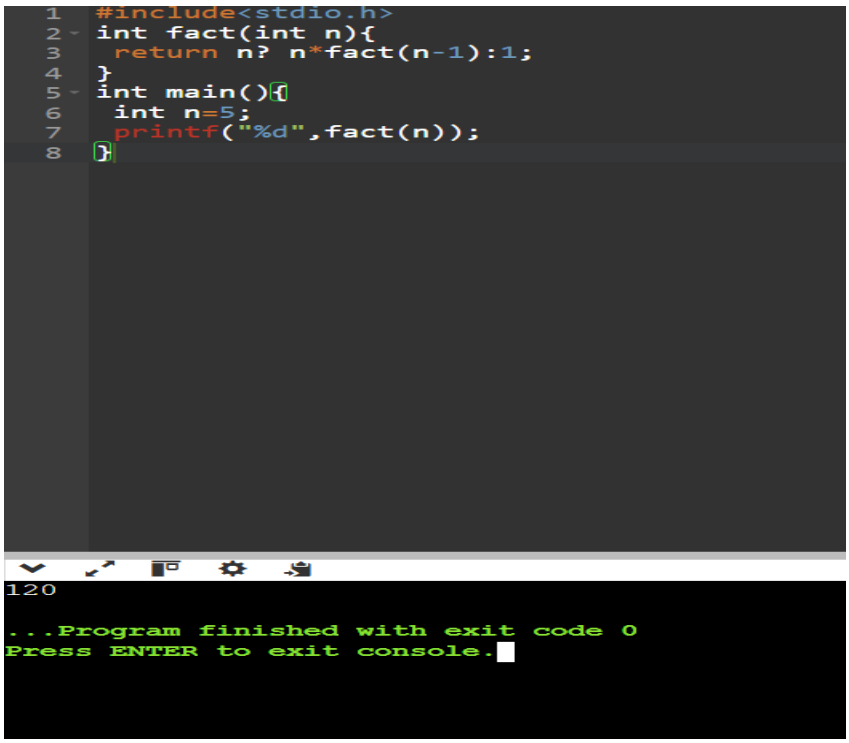
# Experiment 3: Factorial (Iterative vs Recursive)

- **Objective:** To compare iterative and recursive factorial computation.
- **Problem Statement:** Write programs to compute factorial using both methods.
- **Sample Input:** n = 5
- **Sample Output:** 120
- **Complexity:** Both O(n)
- **Explanation:** Highlights recursion depth vs iterative loops.

**Code :**

```
#include<stdio.h>
int fact(int n){
 return n? n*fact(n-1):1;
}
int main(){
 int n=5;
 printf("%d",fact(n));
}
```

Output:




# Experiment 4: Fibonacci Series

- **Objective:** To analyze recursive vs iterative Fibonacci generation.
- **Problem Statement:** Generate first n Fibonacci numbers.
- **Sample Input:** n = 6
- **Sample Output:** [0, 1, 1, 2, 3, 5]
- **Complexity:** Iterative O(n), Recursive naïve O(2n), Recursive with memoization O(n)
- **Explanation:** Illustrates exponential recursion vs efficient iteration.

**Code :**

```
#include<stdio.h>

int main(){

 int n=6,a=0,b=1,c,i;

 for(i=0;i<n;i++){

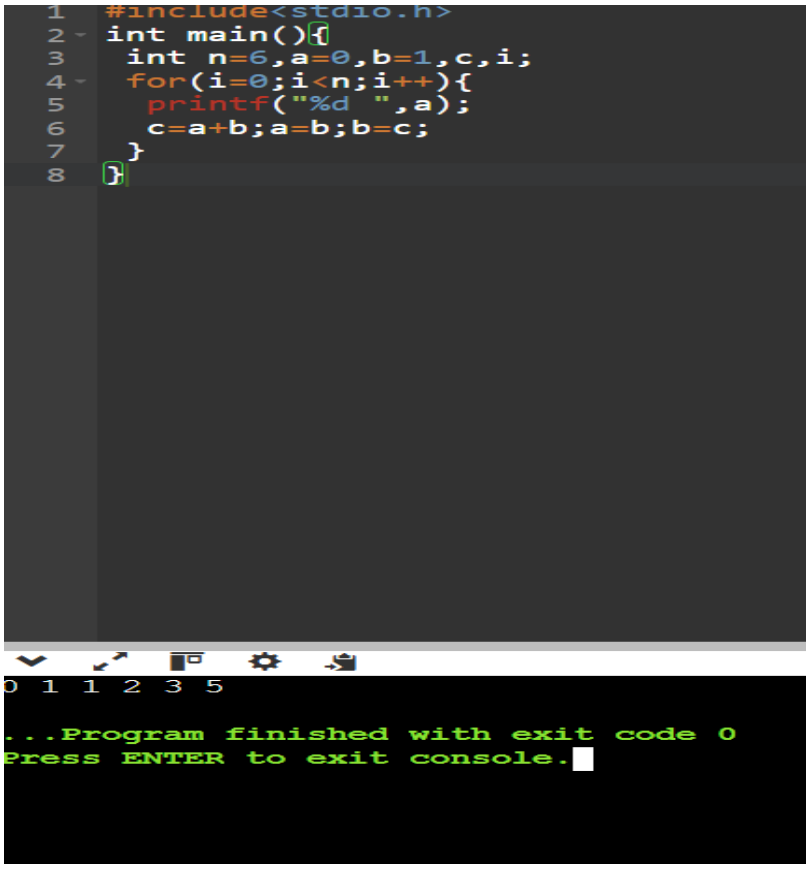
  printf("%d ",a);

  c=a+b;a=b;b=c;

 }

}
```

Output:



# Experiment 5: Matrix Multiplication

- **Objective:** To analyze nested loop complexity.
- **Problem Statement:** Multiply two matrices using loops.
- **Sample Input:** A=[[1,2],[3,4]], B=[[5,6],[7,8]]
- **Sample Output:** [[19,22],[43,50]]
- **Complexity:** O(n3)
- **Explanation:** Shows cubic growth due to nested loops.

**Code :**

```
#include<stdio.h>

int main(){

 int a[2][2]={{1,2},{3,4}},

    b[2][2]={{5,6},{7,8}},

    c[2][2]={0},i,j,k;

 for(i=0;i<2;i++)

  for(j=0;j<2;j++)

   for(k=0;k<2;k++)

    c[i][j]+=a[i][k]*b[k][j];

 for(i=0;i<2;i++){

  for(j=0;j<2;j++)printf("%d ",c[i][j]);

  printf("\n");

 }

}
```

Output :

```c
#include<stdio.h>
int main(){
 int a[2][2]={{1,2},{3,4}},
     b[2][2]={{5,6},{7,8}},
     c[2][2]={0},i,j,k;
 for(i=0;i<2;i++)
  for(j=0;j<2;j++)
   for(k=0;k<2;k++)
    c[i][j]+=a[i][k]*b[k][j];
 for(i=0;i<2;i++){
  for(j=0;j<2;j++)printf("%d ",c[i][j]);
  printf("\n");
 }
}
```

```
19 22
43 50


...Program finished with exit code 0
Press ENTER to exit console.
```

# Experiment 6: Merge Sort

- **Objective:** To analyze divide-and-conquer sorting.
- **Problem Statement:** Implement merge sort.
- **Sample Input:** [38, 27, 43, 3, 9, 82, 10]
- **Sample Output:** [3, 9, 10, 27, 38, 43, 82]
- **Complexity:** Always O(nlog n)
- **Explanation:** Demonstrates guaranteed efficiency of divide-and-conquer.

**Code :**

```
#include<stdio.h>
void ms(int a[],int l,int r){
 if(l>=r)return;
 int m=(l+r)/2,i=l,j=m+1,k=0,t[20];
 ms(a,l,m);ms(a,m+1,r);
 while(i<=m&&j<=r)
  t[k++]=a[i]<a[j]?a[i++]:a[j++];
 while(i<=m)t[k++]=a[i++];
 while(j<=r)t[k++]=a[j++];
 for(i=l,k=0;i<=r;i++)a[i]=t[k++];
}
int main(){
 int a[]={38,27,43,3,9,82,10},i;
 ms(a,0,6);
 for(i=0;i<7;i++)printf("%d ",a[i]);
}
```

Output :

```c
#include<stdio.h>
void ms(int a[],int l,int r){
 if(l>=r)return;
 int m=(l+r)/2,i=l,j=m+1,k=0,t[20];
 ms(a,l,m);ms(a,m+1,r);
 while(i<=m&&j<=r)
  t[k++]=a[i]<a[j]?a[i++]:a[j++];
 while(i<=m)t[k++]=a[i++];
 while(j<=r)t[k++]=a[j++];
 for(i=l,k=0;i<=r;i++)a[i]=t[k++];
}
int main(){
 int a[]={38,27,43,3,9,82,10},i;
 ms(a,0,6);
 for(i=0;i<7;i++)printf("%d ",a[i]);
}
```

```
3 9 10 27 38 43 82

...Program finished with exit code 0
Press ENTER to exit console.
```

# Experiment 7: Quick Sort

- **Objective:** To analyze recursive quick sort.
- **Problem Statement:** Implement quick sort.
- **Sample Input:** [10, 7, 8, 9, 1, 5]
- **Sample Output:** [1, 5, 7, 8, 9, 10]
- **Complexity:** Ω: O(nlog n), O: O(n2), Θ: Θ(nlog n)
- **Explanation:** Shows how pivot choice affects performance.

**Code :**

```
#include<stdio.h>
void q(int a[],int l,int h){
 if(l>=h)return;
 int i=l,j=h,p=a[h],t;
 while(i<j){
  while(a[i]<p)i++;
  while(a[j]>=p&&j>i)j--;
  if(i<j){t=a[i];a[i]=a[j];a[j]=t;}
 }
 t=a[i];a[i]=a[h];a[h]=t;
 q(a,l,i-1);q(a,i+1,h);
}
int main(){
 int a[]={10,7,8,9,1,5},i;
 q(a,0,5);
 for(i=0;i<6;i++)printf("%d ",a[i]);
}
```

Output :

```
#include<stdio.h>
void ms(int a[],int l,int r){
 if(l>=r)return;
 int m=(l+r)/2,i=l,j=m+1,k=0,t[20];
 ms(a,l,m);ms(a,m+1,r);
 while(i<=m&&j<=r)
  t[k++]=a[i]<a[j]?a[i++]:a[j++];
 while(i<=m)t[k++]=a[i++];
 while(j<=r)t[k++]=a[j++];
 for(i=l,k=0;i<=r;i++)a[i]=t[k++];
}
int main(){
 int a[]={38,27,43,3,9,82,10},i;
 ms(a,0,6);
 for(i=0;i<7;i++)printf("%d ",a[i]);
}
```

```
3  9  10  27  38  43  82

...Program finished with exit code 0
Press ENTER to exit console.
```

# Experiment 8: Tower of Hanoi

- **Objective:** To analyze exponential recursion.
- **Problem Statement:** Solve Tower of Hanoi for n disks.
- **Sample Input:** n = 3
- **Sample Output:** Sequence of moves.
- **Complexity:** O(2n)
- **Explanation:** Illustrates exponential growth in recursive calls.

**Code :**

```
#include<stdio.h>

void h(int n,char a,char b,char c){

 if(!n)return;

 h(n-1,a,c,b);

 printf("%c -> %c\n",a,c);

 h(n-1,b,a,c);

}

int main(){

 h(3,'A','B','C');

}
```

Output :

```c
#include<stdio.h>
void h(int n,char a,char b,char c){
 if(!n)return;
 h(n-1,a,c,b);
 printf("%c -> %c\n",a,c);
 h(n-1,b,a,c);
}
int main(){
 h(3,'A','B','C');
}
```

```
A -> C
B -> A
B -> C
A -> C


..Program finished with exit code 0
Press ENTER to exit console.
```

# Experiment 9: GCD (Euclidean Algorithm)

- **Objective:** To analyze recursive GCD.
- **Problem Statement:** Implement Euclidean algorithm.
- **Sample Input:** a=48, b=18
- **Sample Output:** 6
- **Complexity:** O(log n)
- **Explanation:** Shows efficient recursion with logarithmic complexity.

**Code :**

```
#include<stdio.h>

int gcd(int a,int b){

 return b?gcd(b,a%b):a;

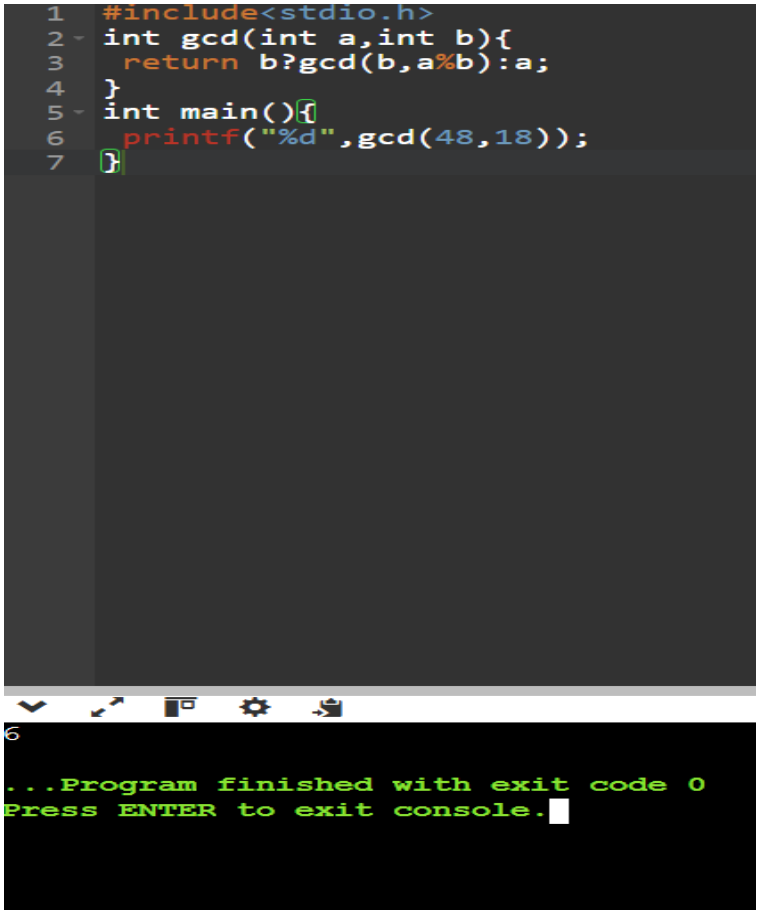
}

int main(){

 printf("%d",gcd(48,18));

}
```

Output :



# Experiment 10: Insertion Sort

- **Objective:** To analyze iterative sorting.
- **Problem Statement:** Implement insertion sort.
- **Sample Input:** [12, 11, 13, 5, 6]
- **Sample Output:** [5, 6, 11, 12, 13]
- **Complexity:** Ω: O(n), O: O(n2), Θ: Θ(n2)
- **Explanation:** Demonstrates input sensitivity (sorted vs unsorted).

**Code :**

```
#include<stdio.h>

int main(){

 int a[]={12,11,13,5,6},i,j,k;

 for(i=1;i<5;i++){

  k=a[i];j=i-1;

  while(j>=0&&a[j]>k)

   a[j+1]=a[j--];

 a[j+1]=k;

 }

 for(i=0;i<5;i++)printf("%d ",a[i]);

}
```

Output :

```c
#include<stdio.h>
int main(){
 int a[]={12,11,13,5,6},i,j,k;
 for(i=1;i<5;i++){
  k=a[i];j=i-1;
  while(j>=0&&a[j]>k)
   a[j+1]=a[j--];
  a[j+1]=k;
 }
 for(i=0;i<5;i++)printf("%d ",a[i]);
}
```

```
5 11 13 5 6

...Program finished with exit code 0
Press ENTER to exit console.
```

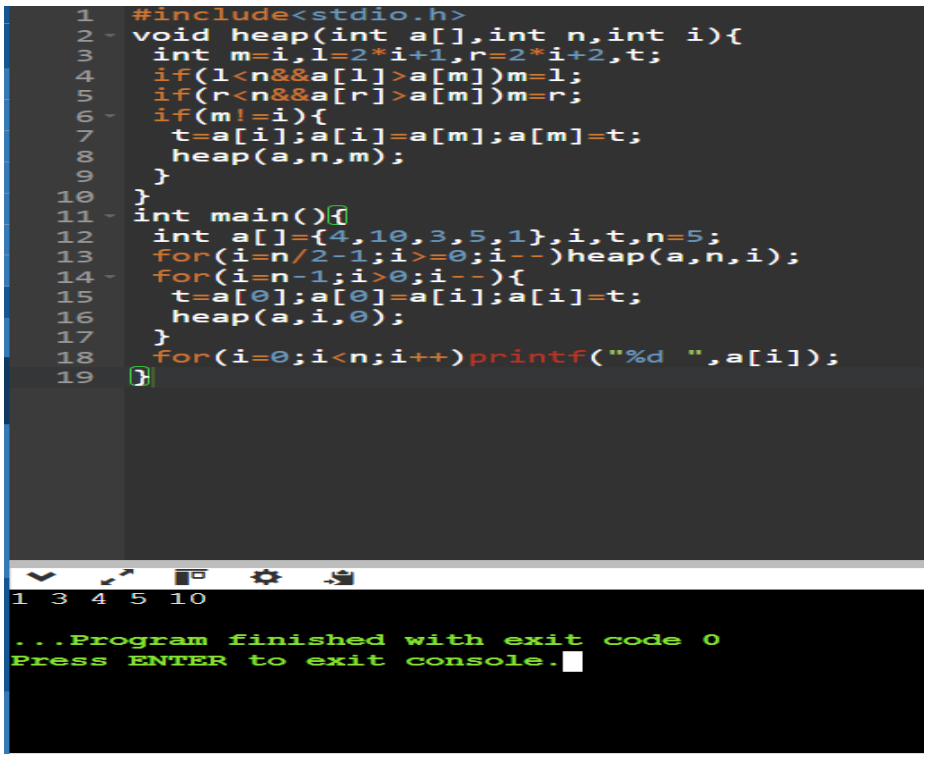
# Experiment 11: Heap Sort

- **Objective:** To analyze heap-based sorting. ● **Problem Statement:** Implement heap sort.
- **Sample Input:** [4, 10, 3, 5, 1]
- **Sample Output:** [1, 3, 4, 5, 10]
- **Complexity:** Always O(nlog n)
- **Explanation:** Consistent performance using heap structure.

**Code :**

```
#include<stdio.h>
void heap(int a[],int n,int i){
 int m=i,l=2*i+1,r=2*i+2,t;
 if(l<n&&a[l]>a[m])m=l;
 if(r<n&&a[r]>a[m])m=r;
 if(m!=i){
  t=a[i];a[i]=a[m];a[m]=t;
  heap(a,n,m);
 }
}
int main(){
 int a[]={4,10,3,5,1},i,t,n=5;
 for(i=n/2-1;i>=0;i--)heap(a,n,i);
 for(i=n-1;i>0;i--){
  t=a[0];a[0]=a[i];a[i]=t;
  heap(a,i,0);
 }
 for(i=0;i<n;i++)printf("%d ",a[i]);
}
```

Output :



# Experiment 12: Counting Inversions

- **Objective:** To analyze modified merge sort.
- **Problem Statement:** Count inversions in an array.
- **Sample Input:** [2, 4, 1, 3, 5]
- **Sample Output:** 3
- **Complexity:** O(nlog n)
- **Explanation:** Shows divide-and-conquer applied beyond sorting.

**Code :**

```
#include<stdio.h>

int main(){

 int a[]={2,4,1,3,5},i,j,c=0;

 for(i=0;i<5;i++)

  for(j=i+1;j<5;j++)

   if(a[i]>a[j])c++;

 printf("%d",c);

}
```

Output :

```c
#include<stdio.h>
int main(){
 int a[]={2,4,1,3,5},i,j,c=0;
 for(i=0;i<5;i++)
  for(j=i+1;j<5;j++)
   if(a[i]>a[j])c++;
 printf("%d",c);
}
```

```
3

...Program finished with exit code 0
Press ENTER to exit console.
```

# Experiment 13: Maximum Subarray Sum

- **Objective:** To compare Kadane's vs divide-and-conquer.
- **Problem Statement:** Find maximum subarray sum.
- **Sample Input:** [-2, -3, 4, -1, -2, 1, 5, -3]
- **Sample Output:** 7
- **Complexity:** Kadane's O(n), Divide & Conquer O(nlog n)
- **Explanation:** Highlights efficiency differences between approaches.

**Code :**

```
#include<stdio.h>

int main(){

 int a[]={-2,-3,4,-1,-2,1,5,-3},i,s=a[0],m=a[0];

 for(i=1;i<8;i++){

  s=s+a[i]>a[i]?s+a[i]:a[i];

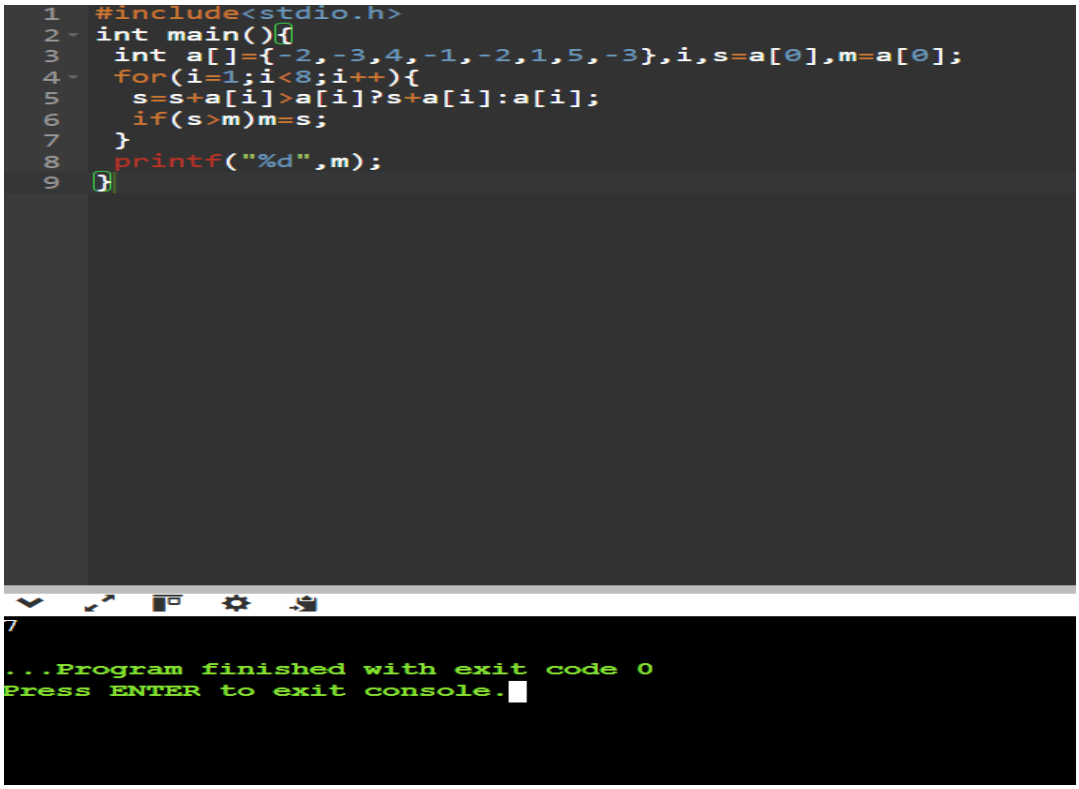
  if(s>m)m=s;

 }

 printf("%d",m);

}
```

Output :



# Experiment 14: Exponentiation

- **Objective:** To compare iterative vs recursive fast power.
- **Problem Statement:** Compute xn.
- **Sample Input:** x=2, n=10
- **Sample Output:** 1024
- **Complexity:** Iterative O(n), Recursive fast power O(log n) ● **Explanation:** Shows how recursion reduces complexity.

**Code :**

```
#include<stdio.h>

int power(int x,int n){

 if(!n)return 1;

 int y=power(x,n/2);

 return n%2?y*y*x:y*y;

}

int main(){

 printf("%d",power(2,10));

}
```

Output :

```c
#include<stdio.h>
int power(int x,int n){
 if(!n)return 1;
 int y=power(x,n/2);
 return n%2?y*y*x:y*y;
}
int main(){
 printf("%d",power(2,10));
}
```

```
1024

...Program finished with exit code 0
Press ENTER to exit console.
```

# Experiment 15: Closest Pair of Points

- **Objective:** To analyze brute force vs divide-and-conquer.
- **Problem Statement:** Find closest pair of points in 2D.
- **Sample Input:** [(1,2), (4,5), (7,8), (3,1)]
- **Sample Output:** Closest pair (1,2)-(3,1), Distance = √5
- **Complexity:** Brute force O(n2), Optimized O(nlog n)
- **Explanation:** Demonstrates quadratic complexity and motivates optimization.

Code :

```
#include<stdio.h>
#include<math.h>
int main(){
 int x[]={1,4,7,3},y[]={2,5,8,1},i,j,a=0,b=1;
 double d,m=1e9;
 for(i=0;i<4;i++)
  for(j=i+1;j<4;j++){
   d=hypot(x[i]-x[j],y[i]-y[j]);
   if(d<m)m=d,a=i,b=j;
  }
 printf("(%d,%d)-(%d,%d) Distance=%.2f",
 x[a],y[a],x[b],y[b],m);
}
```

Output :

```c
#include<stdio.h>
#include<math.h>
int main(){
 int x[]={1,4,7,3},y[]={2,5,8,1},i,j,a=0,b=1;
 double d,m=1e9;
 for(i=0;i<4;i++)
  for(j=i+1;j<4;j++){
   d=hypot(x[i]-x[j],y[i]-y[j]);
   if(d<m)m=d,a=i,b=j;
  }
 printf("(%d,%d)-(%d,%d) Distance=%.2f",
 x[a],y[a],x[b],y[b],m);
}
```

```
(1,2)-(3,1) Distance=2.24

...Program finished with exit code 0
Press ENTER to exit console.
```